



TECHNICAL DOCUMENT · PROOF OF CONCEPT

Architecture, Implementation and Controls

A secure, multilingual multi-agent assistant for SME business performance monitoring, deployed as a serverless proof of concept on AWS (Singapore).



Event	NUS-ISS Show Me Your Agents Hackathon
Selected problem	Business Performance Monitoring
Team	Yiting · Sonne · Tiffany · Malar
Version / date	1.0 · 27 September 2026 (Singapore)
Deployed POC	https://poc.d1w9q0ck4cdi9s.amplifyapp.com (invite-only, Cognito sign-in)
Source snapshot	BizzyBackend main@82e3836 · BizzyFrontend main@993848f + PR #9 · BizzyData validated dataset

CONTENTS

1	Executive summary	9	CI/CD and deployment
2	Problem and scope	10	Operations and cost
3	Solution architecture	11	Testing and measured results
4	Agent hive implementation	12	Demo walkthrough
5	Data layer	13	Known limitations and next steps
6	Governance: Guard, approvals, audit	14	Team and ownership
7	Security controls	A	API reference
8	Frontend implementation	B	Sources and conventions

All business data in this document is synthetic. Account identifiers, Cognito identifiers and user e-mail addresses are intentionally omitted.

1 Executive summary

SME owners receive sales reports, accounting data, inventory records and customer enquiries from separate systems, and spend considerable time reconciling them before they understand what needs attention. **BizzyBee AI** lets the owner ask one business question in their language and returns a single, evidence-backed answer that combines the relevant specialist views, recommends next steps and keeps every commitment under human approval.

The proof of concept implements the full planned loop: **detect** an issue on the dashboard, **route** the question to the right specialist agents, **correlate** their evidence, **explain** the likely cause, **recommend** actions, **classify** each action with a deterministic GREEN / AMBER / RED policy, **record** the workflow in a durable audit store and let the owner **approve or reject** AMBER drafts. Nothing is executed automatically: there is no payment, messaging or purchasing tool in the system.

8 agents

Queen, Sales, Inventory, Finance,
Customer, Advisor, Guard and Audit
Bees behind one API contract

498,535 records

21 synthetic CSV datasets, fixed
reporting date 23 Sep 2026,
reproducible byte-for-byte

0.29 s

local backend time for the flagship
cross-agent question (baseline
assumption: ~10 min manual review)

100% controlled

6/6 dangerous or unknown action
types blocked even when labelled
GREEN; every AMBER action needs
approval

26 resources

serverless AWS runtime managed by
Terraform; no VPC, NAT, load balancer
or database server

133 + 28 tests

backend tests passed (41 opt-in
benchmarks skipped) plus 28 frontend
unit tests and contract checks

Key design decisions

- **Deterministic numbers, bounded AI.** Every SGD amount, count and date is computed with SQL (DuckDB) over the packaged data. Amazon Bedrock (Nova Micro) is used only as a routing fallback whose output must be a set of known agent names; invalid or failed output becomes a visible clarification, never a guess.
- **Policy before persuasion.** Guard Bee classifies actions by an allowlist, not by the risk label an agent attaches. Unknown and RED actions are blocked; AMBER actions are recorded as pending until the owner decides.
- **Serverless and cheap to idle.** One FastAPI application runs in a Lambda container behind API Gateway with a Cognito JWT authorizer; audit and approvals live in DynamoDB. The planning allowance is about USD 13 per month under the stated POC usage.
- **Traceability.** Each workflow stores the routed agents, statuses, evidence count, every proposed action with its policy and state, the full agent results and an append-only list of approval decisions guarded by a revision number.

Current status (27 Sep 2026). The runtime is deployed and verified (health, readiness, 401 without a token, exact-origin CORS, DynamoDB PITR). The frontend is published on Amplify and four team members were invited; only the owner is in the `approvers` group. Bedrock routing was enabled after one approved paid smoke test. Continuous deployment is intentionally disabled; releases are owner-operated. A frontend compatibility fix for the audit trace view is open as PR #9 (section 13).

2 Problem and scope

2.1 Problem statement

Business Performance Monitoring: SME owners receive business information from multiple sources, including sales reports, accounting systems, inventory records, customer enquiries and operational updates. Because the information is scattered, owners spend considerable time reviewing data before understanding overall business performance and identifying issues that require immediate attention.

2.2 Gaps addressed

Gap	How the POC responds
Fragmentation	One dashboard and one question box over sales, inventory, finance and customer data; Queen Bee combines only the specialists a question needs.
Knowledge	Advisor Bee links findings across domains (for example lost revenue for a product versus its unfulfilled demand) and states that this is correlation, not proof of causation.
Language	Questions and summaries in English and Simplified Chinese; SGD values, product IDs, invoice IDs, dates and approval states stay structured and identical across languages.
Action	Each specialist proposes typed next steps with a reason, parameters and expected impact, not just charts.
Trust	Every recommendation carries evidence (metric, value, period, source file); Guard Bee enforces approval; Audit Bee records who decided what and why.

2.3 Staged scope and delivery status

Milestone	Planned scope	Delivered
Week 1	Architecture, common agent/API contract and health indicators; synthetic sales, finance, inventory and customer data; Sales, Customer, Finance and Inventory Bees with FastAPI endpoints.	DONE Contract (AgentResponse), BizzyData (21 CSVs), four specialists, REST endpoints, health score and alert feed.
Week 2	Queen orchestration and Advisor reasoning; Amazon Bedrock; Guard approval rules and Audit traceability; all eight agents integrated; multilingual dashboard.	DONE All eight roles integrated. SCOPED Bedrock is limited to routing fallback; localised output is English and Chinese.
Final	Deploy the MVP on AWS and demonstrate detect → coordinate → correlate → explain → recommend → approve → audit on the flagship scenario.	DONE Deployed to AWS Singapore; flagship scenario verified end-to-end (section 12). OPEN CD disabled, audit list endpoint pending.

2.4 Boundaries of the MVP

- Synthetic data only; no connection to real POS, accounting, CRM, e-mail or WhatsApp systems.
- No autonomous execution: approvals are *recorded*; no payment, purchase order or customer message is sent.
- Single business, invite-only users; tenant isolation, data retention policies and a WORM audit archive are future work.

3 Solution architecture

3.1 Conceptual architecture (planning reference)

The team agreed the conceptual architecture below on 23 September 2026. It is kept here as the product-level reference; the implemented POC keeps the same layers and agent roles but makes several deliberate infrastructure substitutions (table 3.2).

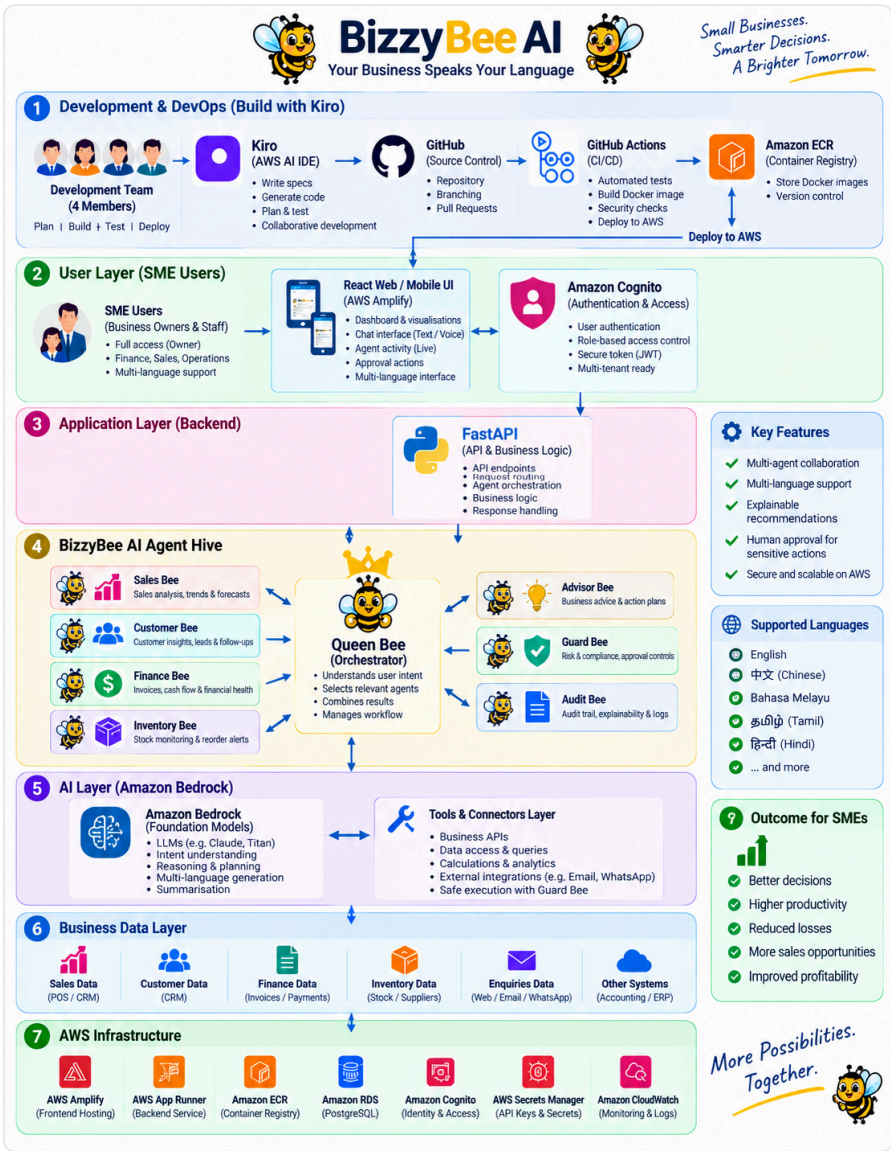


Figure 1. Conceptual architecture from the Master Team Reference (23 Sep 2026). App Runner, RDS and Secrets Manager shown here were not used in the POC.

3.2 Planned versus implemented

Layer	Planned	Implemented POC	Reason
Frontend	React web on AWS Amplify	React 19 + Vite static build on Amplify, manual deployment, no Git connection	One deployment controller; static hosting only
Backend compute	FastAPI on App Runner	FastAPI in a Lambda container (Mangum) behind API Gateway HTTP API	App Runner is not available to new customers; Lambda has no idle cost

Layer	Planned	Implemented POC	Reason
Business data	PostgreSQL on RDS	Packaged CSVs queried with DuckDB inside the image; fixed as-of date 2026-09-23	Read-only synthetic data needs no database server; reproducible results
Audit and approvals	RDS tables	DynamoDB single-item workflow records with conditional updates	Serverless, pay-per-request, PITR and deletion protection
AI layer	Bedrock for intent, reasoning, multilingual generation	Bedrock Nova Micro (APAC inference profile) for routing fallback only	Keeps numbers and explanations deterministic and testable; negligible cost
Secrets	Secrets Manager	None required: no third-party API keys; IAM role credentials only	Removes a class of secrets entirely
Identity	Cognito	Cognito user pool, hosted UI, authorization code + PKCE, <code>approvers</code> group	As planned
Delivery	GitHub Actions → ECR → AWS	Actions CI on every PR; deploy jobs disabled; owner-operated release	GitHub Free private repos cannot enforce environment reviewers

3.3 Request flow

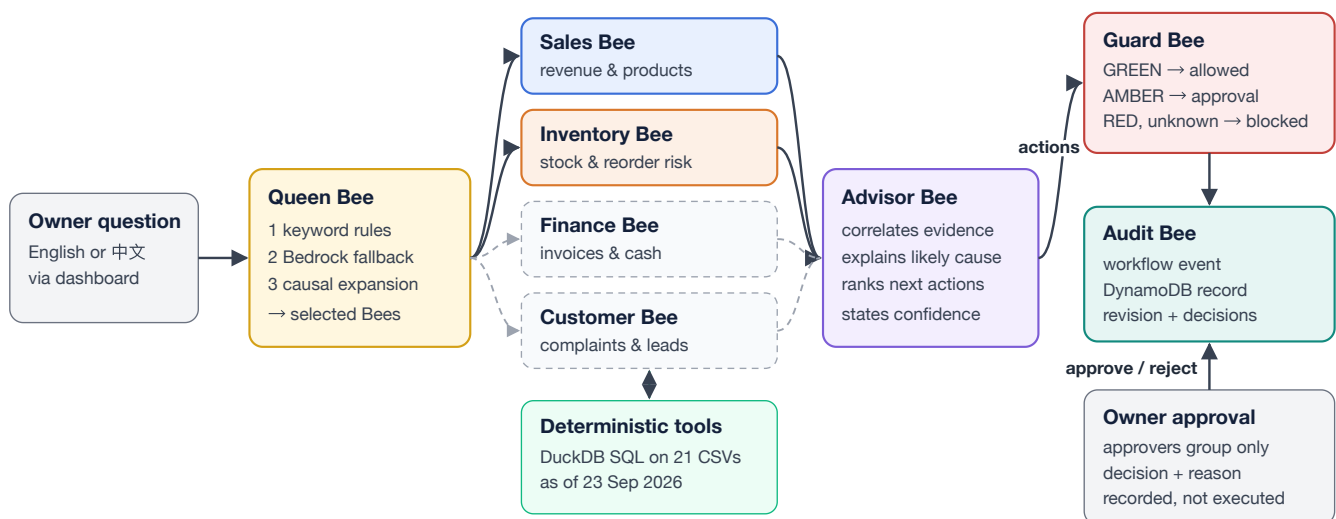


Figure 2. Request flow. Solid paths are the flagship question “Why did sales fall this week?”; dashed specialists run only when the question needs them.

- Ask.** The dashboard sends `POST /api/v1/query` with the question and language, using the signed-in user’s Cognito access token.
- Route.** Queen Bee first applies deterministic English and Chinese keyword rules. If no rule matches, it asks Bedrock Nova Micro to choose from the known specialists; if the model is disabled, fails or returns anything else, the response asks the user to clarify. If Sales proposes `investigate_stock_availability` for a decline question, Queen adds Inventory automatically (causal expansion).
- Analyse.** Each selected specialist runs SQL over the packaged datasets and returns an `AgentResponse` with a summary, typed evidence, confidence and recommended actions.
- Correlate.** Advisor Bee combines the results, for example matching PRD001’s lost revenue (SGD 12,600) with its unfulfilled demand value (SGD 12,600), and proposes a prioritised action.
- Classify.** Guard Bee evaluates every proposed action and returns `allowed`, `approval_required` or `blocked` for the workflow.

6. **Record.** Audit Bee writes the workflow event before the response is returned; if the write fails, the request fails rather than reporting an unaudited success.
7. **Decide.** The owner can approve or reject each pending AMBER action with a reason; the decision is appended to the same record with a revision check.

3.4 AWS deployment architecture

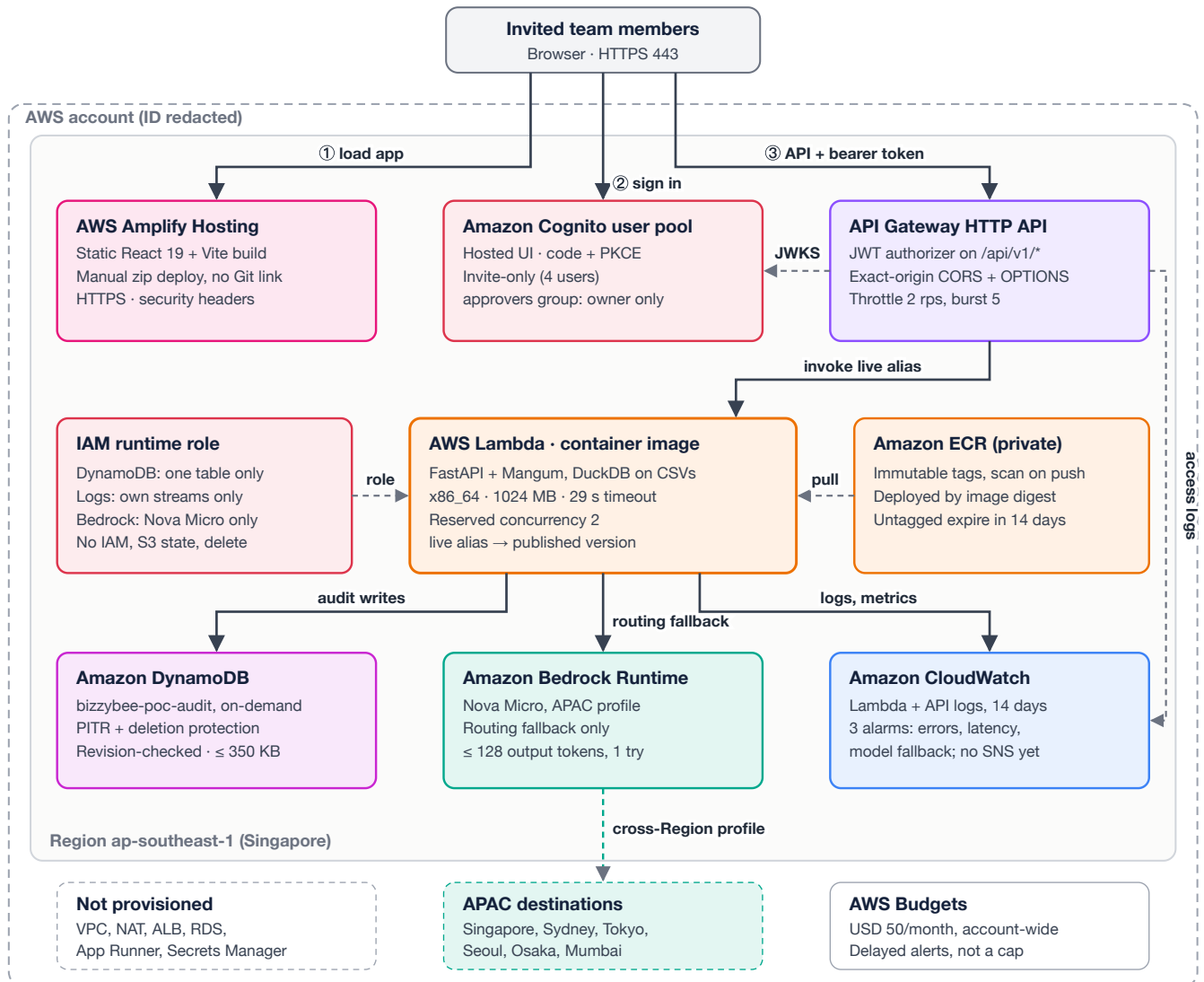


Figure 3. Deployed AWS topology (redrawn from the team’s deployment diagram and the Terraform runtime root). All managed services are reached over HTTPS; no customer-managed network resources exist.

- **Three client paths.** The browser loads static assets from Amplify, signs in through the Cognito hosted UI (authorization code with PKCE) and calls the API with a bearer access token. API Gateway validates the JWT against Cognito before invoking the Lambda *live* alias.
- **One application.** A single FastAPI app serves all specialist, query, audit and approval routes. The dataset is baked into the image with a fixed reporting date, so every environment answers identically.
- **Low blast radius.** Reserved concurrency 2 and API throttling (2 requests/s, burst 5) cap cost and load; the runtime role can only read and write one DynamoDB table, write its own logs and invoke the approved Nova Micro profile.
- **Why no VPC.** Every dependency is an AWS-managed endpoint reached with SigV4 over HTTPS, so a VPC, NAT gateway or load balancer would add cost without reducing exposure for this POC.

4 Agent hive implementation

4.1 Roles and implementation

Bee	Responsibility	Implementation	Proposed actions (policy)
Queen	Interpret the question, select and run the relevant specialists, combine results.	<code>orchestration/queen.py</code> : English and Chinese keyword rules, Bedrock <code>classify</code> fallback, causal expansion of Sales → Inventory.	None (routing only)
Sales	Week-on-week revenue, product and channel decomposition, volume versus price/mix effect.	<code>agents/sales.py</code> : SQL over sales orders and items; fixed 7-day windows ending 23 Sep.	<code>investigate_stock_availability</code> GREEN
Inventory	Stock status, days of cover versus supplier lead time, unfulfilled demand, reorder suggestions.	<code>agents/inventory.py</code> : stock, demand, receipts and supplier tables; provisional quantities when inbound is unknown.	<code>verify_inbound_quantity</code> , <code>prepare_purchase_order</code> AMBER
Finance	Overdue invoices, receivables, profitability and cash-flow reports from the ledger.	<code>agents/finance.py</code> , <code>api/finance_reports.py</code> : ledger-based figures with source and period on every value.	<code>prepare_invoice_reminders</code> AMBER
Customer	Complaints, enquiries, leads and follow-up opportunities.	<code>agents/specialists.py</code> : complaint categories, defect signals, unanswered enquiries.	<code>prepare_customer_follow_up</code> , <code>review_supplier_quality_defect</code> AMBER
Advisor	Correlate specialist evidence, explain the likely cause, prioritise actions.	Deterministic synthesis with evidence validation; explicitly reports correlation, not proof of causation.	<code>prioritise_revenue_recovery</code> GREEN
Guard	Apply GREEN / AMBER / RED controls to every proposed action.	<code>security/guard.py</code> : allowlists, parameter validation, workflow aggregation (section 6).	Decides; never proposes
Audit	Record evidence, actions, decisions and outcomes.	<code>audit/store.py</code> : workflow event, DynamoDB conditional updates, read isolation.	Records; never executes

4.2 Common agent contract

Every specialist and the Advisor return the same Pydantic model, which is also pinned in the frontend contract check. Structured evidence is what makes cross-agent correlation, language-independent values and auditing possible.

```

class Evidence(BaseModel):
    metric: str; value: Any; unit: str | None; period: str | None
    source: str | None; dimensions: dict[str, str] # e.g. product_id, channel

class RecommendedAction(BaseModel):
    type: str # must be on a Guard allowlist
    risk_level: RiskLevel = GREEN # GREEN | AMBER | RED (advisory only)
    reason: str | None; parameters: dict; expected_impact: dict

class AgentResponse(BaseModel):
    agent: str; status: success | partial | failed; summary: str
    evidence: list[Evidence]; confidence: float [0, 1]
    recommended_actions: list[RecommendedAction]

class QueryResponse(BaseModel):
    workflow_id: str; invoked_agents: list[str]
    specialist_results: list[AgentResponse]; advisor_result: AgentResponse
    guard_decision: "allowed" | "approval_required" | "blocked"; approval_required: bool

```

4.3 Deterministic first, AI where it is safe

Concern	Approach
Business figures	Always SQL/Python over the packaged data. The model never produces or edits an SGD value, quantity, ID or date.
Routing	Keyword rules first (zero cost, fully testable). Only unmatched questions reach Bedrock via the Converse API with 128 output tokens, 2 s connect / 6 s read timeouts and one attempt. The reply must be a unique list of allowed agent names; anything else becomes a clarification.
Explanations	Deterministic, templated summaries in English and Simplified Chinese, generated from the same evidence objects.
Failure behaviour	Model disabled, throttled, denied or malformed → visible <code>partial</code> status and a clarification prompt; dashboard views stay available.
Verified model call	One approved paid smoke test routed a stock question to <code>["inventory"]</code> in 507 ms using 74 input and 7 output tokens. At published Singapore rates, 1,000 routing calls cost about USD 0.05.
Privacy	No raw prompt logging. In the deployed POC the question text is not stored in the audit record (<code>BIZZY_RECORD_QUESTIONS=false</code>); evidence and actions are.

5 Data layer

The **BizzyData** repository contains a deterministic generator and validator for a synthetic SME: 21 CSV datasets with 498,535 records (about 59.6 MB) covering products, sales orders and items, stock, receipts, suppliers, purchase orders, invoices, payments, the accounting ledger, expenses, customers, complaints and enquiries. Regeneration in a clean copy matches the committed files byte-for-byte, and a SHA-256 manifest is packaged with the backend image. The reporting date is fixed at **23 September 2026**, so every run and every environment returns the same answers.

Domain	Future source	Quality / privacy concern and control
Sales transactions	POS / CRM	Incomplete mappings or stale data → every value carries its period and source file.

Domain	Future source	Quality / privacy concern and control
Finance / invoices	Accounting system	Sensitive financial data → least privilege, read-only reports, no payment tool.
Inventory / suppliers	Inventory system	Outdated stock or lead times → provisional order quantities until inbound is confirmed.
Customer enquiries	Web, e-mail, WhatsApp	PII and multilingual free text → synthetic only in the POC; minimise and mask in production.

Flagship scenario embedded in the data

Signal	Value (as of 23 Sep 2026)
Weekly revenue	SGD 70,000.00 (10–16 Sep) → SGD 57,400.00 (17–23 Sep), –18.0% , –SGD 12,600.00
Source of the decline	Product A – Premium Blender (PRD001): 100% of the decline, no sales since 16 Sep; volume effect –SGD 12,600, price/mix effect SGD 0
Stock position	PRD001 at 0 units (reorder level 30); 126 units of demand unfulfilled ≈ SGD 12,600 lost sales; next delivery 28 Sep with quantity not confirmed
Wider inventory risk	1 out of stock, 20 may run out before restock, 29 healthy; 21 products to reorder (442 units suggested)
Finance baseline	528 overdue invoices, SGD 209,354 outstanding
Dashboard health	Business health 59/100; 10 alerts (2 critical, 5 warning, 3 info)

6 Governance: Guard, approvals and audit

6.1 Action policy

Level	Meaning	Examples in the POC	Enforcement
GREEN	Analysis, KPI calculation, anomaly detection and explanation may run automatically.	<code>investigate_stock_availability</code> , <code>review_inventory_analysis</code> , <code>prioritise_revenue_recovery</code>	Allowed only if the type is on <code>READ_ONLY_TYPES</code> .
AMBER	Drafts that would create a business commitment require an authorised human.	<code>prepare_purchase_order</code> , <code>verify_inbound_quantity</code> , <code>prepare_customer_follow_up</code> , <code>prepare_invoice_reminders</code>	On <code>APPROVAL_TYPES</code> or labelled <code>AMBER</code> → <code>approval_required</code> ; stored as <i>pending</i> .
RED	Autonomous transfers, supplier payments, bank-detail changes, deleting financial records, privileged-access changes.	<code>supplier_payment</code> , <code>bank_transfer</code> , <code>change_bank_details</code> , <code>delete_financial_records</code>	Labelled <code>RED</code> or <i>not on any allowlist</i> → <code>blocked</code> .

```
def action_decision(action) -> str:
    if action.risk_level == RED or action.type not in READ_ONLY_TYPES | APPROVAL_TYPES:
        return "blocked" # fail closed: unknown means blocked
    if _invalid_controlled_action(action.type, action.parameters):
        return "blocked" # e.g. purchase order without product, supplier, qty
    > 0
    if action.type in APPROVAL_TYPES or action.risk_level == AMBER:
        return "approval_required" # a GREEN label cannot downgrade a controlled type
    return "allowed"
# workflow decision: any blocked -> blocked; else any approval_required -> approval_required; else
allowed
```

6.2 Approval workflow

- Only members of the Cognito `approvers` group (the owner) can call `POST /api/v1/actions/{action_id}/approve` or `/reject`; the UI asks for a reason (up to 1,000 characters).
- The decision is appended to the workflow record with a DynamoDB conditional update on its `revision`, so two concurrent decisions cannot overwrite each other; repeating the same decision by the same actor is idempotent.
- Approving changes the action state from *pending* to *approved* and records actor, time and reason. No executor exists: approval authorises a draft, it does not send, pay or order anything.

Approval gate

Authorised decisions are recorded only. No external business action is executed.
Decisions are recorded only. No payment, message or purchase is executed.

Decision reason

- investigate_stock_availability: read_only
- verify_inbound_quantity: pending Approve draft Reject draft
- prioritise_revenue_recovery: read_only

Figure 4a. Approval gate with one pending AMBER action.

Approval gate

Authorised decisions are recorded only. No external business action is executed.
Decisions are recorded only. No payment, message or purchase is executed.

Decision reason

- investigate_stock_availability: read_only
- verify_inbound_quantity: approved
- prioritise_revenue_recovery: read_only

Figure 4b. After the owner approves with a reason.

6.3 Audit record

Field	Content
<code>workflow_id</code> , <code>user</code> , <code>created_at</code>	UUID, Cognito subject of the caller (never taken from the request body), UTC timestamp.
<code>agents</code> , <code>statuses</code>	Agents that produced results and their <code>success</code> / <code>partial</code> / <code>failed</code> status.
<code>decision</code> , <code>evidence_count</code>	Guard workflow decision and number of evidence items (55 for the flagship question).
<code>actions[]</code>	<code>action_id</code> (<code>workflow:index</code>), agent, type, risk level, reason, parameters, expected impact, Guard policy and state (<i>read_only</i> , <i>pending</i> , <i>approved</i> , <i>rejected</i> , <i>blocked</i>).
<code>results[]</code>	Full <code>AgentResponse</code> of every specialist and the Advisor, so evidence can be re-inspected later.
<code>revision</code> , <code>decisions[]</code>	Optimistic-concurrency counter and the append-only list of approval decisions.

Records larger than 350 KB fail closed instead of silently truncating evidence. A workflow can be read only by the user who created it or by an approver; anyone else receives 404, and a conflicting concurrent decision receives 409. The table has point-in-time recovery and deletion protection; it is not a WORM archive, and CloudTrail management events are the AWS-level audit boundary.

7 Security controls

Area	Control as implemented
Authentication	Cognito hosted UI with authorization code + PKCE (<code>oidc-client-ts</code>). Tokens are kept in memory only, so a reload requires sign-in; access tokens expire after 15 minutes.
API authorization	API Gateway JWT authorizer on <code>ANY /api/v1/{proxy+}</code> . The backend verifies the signature, issuer, client and <code>token_use=access</code> again and binds identity to the token subject. ID tokens are rejected.
Approval rights	Owner-only <code>approvers</code> group; approval endpoints return 403 for other users; workflow reads are isolated per user.
CORS	Exact Amplify origin only, methods GET/POST/OPTIONS, max-age 300 s. A separate unauthenticated <code>OPTIONS</code> route answers preflight; untrusted origins receive no allow-origin header.
Least privilege	Runtime role: Get/Put/Update on one table, own log streams, InvokeModel on the approved profile and destination ARNs. No IAM, Cognito admin, S3 state or delete permissions. Deployment roles are scoped to one ECR repository, one Lambda function and one Amplify branch.
Fail closed	In the POC configuration the app refuses to start without Cognito settings, a durable audit table and HTTPS origins. An audit write failure returns an error instead of an unaudited answer.
Supply chain	SHA-pinned GitHub Actions; Trivy scan must pass with no HIGH/CRITICAL findings (FastAPI 0.141.1, Starlette 1.3.1 and PyJWT 2.15.0 were upgraded to clear findings); immutable ECR tags with scan on push; <code>npm audit</code> reported zero vulnerabilities.
Secrets and state	No API keys anywhere; only public identifiers in frontend build variables. Terraform state is in a private, encrypted, versioned S3 bucket with a TLS-only policy and native locking.
Abuse and cost limits	API throttle 2 requests/s (burst 5), Lambda reserved concurrency 2, Bedrock output capped at 128 tokens with a single attempt, account budget alerts.

8 Frontend implementation

The dashboard is a React 19 + TypeScript 6 single-page app built with Vite 8 and linted with oxlint. It is organised into six areas that follow the owner's journey: **Overview** (health score, KPIs, priority alerts, Sales/Inventory/Finance panels), **Ask Bizzy**, **Agent hive** (live state of all eight Bees and a per-Bee workspace), **Analytics** (query-derived metrics and SQL evidence), **Governance** (Advisor guidance, Guard evaluation and the approval gate) and **Audit trail**.

Concern	Implementation
API client	Typed contracts in <code>src/types/contracts.ts</code> ; every request adds the bearer token and a 30 s timeout.
Request scheduler	<code>src/api/transport.ts</code> : at most 2 concurrent requests with 600 ms spacing, matching the API Gateway throttle; read-only GETs retry on 429/503.
Contract safety	<code>test:contract</code> checks the pinned backend OpenAPI routes, authentication requirements and required fields.
Honest state	The hive never invents progress: specialists show <i>Waiting</i> until results arrive, Audit shows <i>Recorded</i> only after the audit record is fetched, and nothing is shown as <i>Executed</i> unless an audit record confirms it.
Audit compatibility	PR #9 maps the stored workflow event returned by <code>GET /audit/{id}</code> to the trace view model and shows the stored per-action Guard policy (section 13).

Concern	Implementation
Language	Language selector on the question; English and Chinese summaries are returned by the backend, and critical values stay structured.

9 CI/CD and deployment

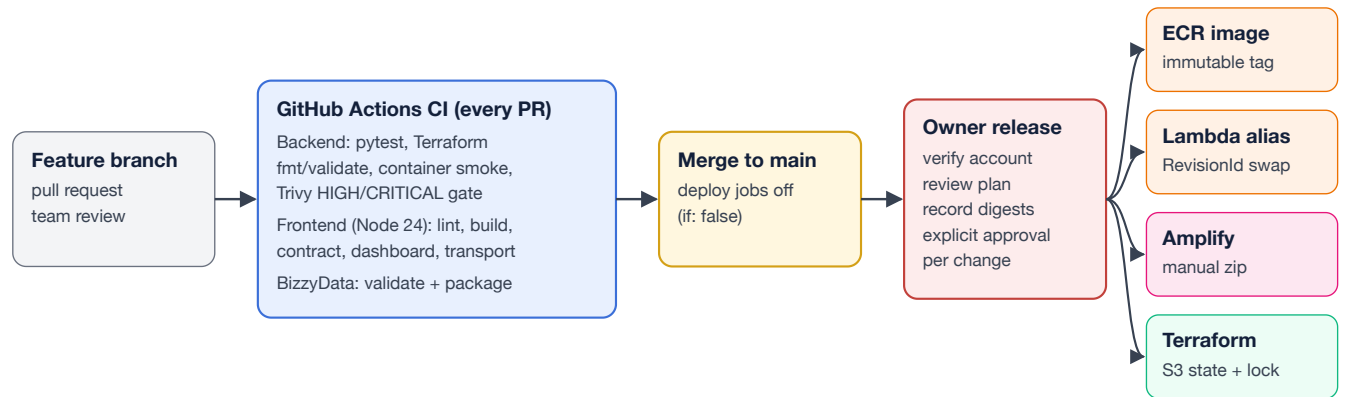


Figure 5. Delivery pipeline. CI runs unprivileged on every pull request; deployment is owner-operated because private repositories on the GitHub Free plan cannot enforce required reviewers on a deployment environment.

9.1 Infrastructure as code

Terraform root	Resources	Applied
<code>infra/bootstrap</code>	Private S3 state bucket (encryption, versioning, public-access block, TLS-only policy) and private ECR repository (immutable tags, scan on push, 14-day untagged expiry).	7 added
<code>infra/runtime</code>	Amplify app and branch; Cognito pool, client, domain and approvers group; Lambda function and <i>live</i> alias; HTTP API, JWT authorizer, routes, stage and throttling; DynamoDB table; log groups and three alarms; runtime and deployment IAM roles.	26 added; later +1 (OPTIONS route) and 2 updated (Bedrock enablement)
<code>infra/budget</code>	Imports and manages the existing account-wide monthly cost budget (USD 50) instead of creating a duplicate.	Budget unchanged

9.2 Release and rollback runbook

- Backend.** Build the Lambda image once, run the tests and container smoke test, scan it with Trivy, push an immutable tag to ECR and record the digest. Update the function code, publish a numbered version, invoke the candidate's `/ready`, then move the *live* alias with a conditional update on its `RevisionId` and re-check HTTPS readiness.
- Frontend.** Build with the public API and Cognito settings, zip `dist/`, create an Amplify manual deployment, poll the job and verify `release.json` and asset hashes.
- Rollback.** Backend: conditionally move the alias back to the recorded previous version (version 1 is kept as the inference-disabled fallback). Frontend: redeploy the retained previous zip with its matching release metadata rather than rebuilding an old commit.

9.3 Deployment history

Change	Verification
Runtime apply: 26 added, 0 changed, 0 destroyed	<code>/health</code> and <code>/ready</code> 200 (reporting date 2026-09-23); <code>/api/v1/me</code> 401 without a token; PITR and deletion protection on; final plan shows no changes.

Change	Verification
Frontend published (Amplify job 3) and four invitations	Every published JS/CSS asset matched its local build hash; Cognito discovery and hosted login returned 200; only the owner is in <code>approvers</code> .
CORS preflight route added (1 added)	Preflight 200 with the exact origin on seven API paths; unauthenticated GET/POST remain 401.
Bedrock enablement (2 updated), version 2 promoted	Candidate readiness 200; <i>live</i> moved from version 1 to 2 with the recorded RevisionId; the team's deployment diagram shows the alias on version 3 after a later code release.

10 Operations and cost

Monitoring

Alarm	Condition
<code>backend-errors</code>	Lambda <code>Errors</code> ≥ 1
<code>backend-duration</code>	Lambda <code>Duration</code> $\geq 25,000$ ms (limit 29 s)
<code>inference-fallback</code>	Custom <code>InferenceFallback</code> ≥ 3

Lambda and API access logs are kept for 14 days. Alarms have no SNS action yet, so someone must watch the CloudWatch console; budget e-mails are not an incident channel.

Monthly planning allowance (USD)

Component	USD
Lambda and HTTP API	< 1
Bedrock (1,000 routing calls)	0.05
Amplify storage and transfer	1
ECR, S3 state, DynamoDB + PITR	3
CloudWatch logs and alarms	2
Cognito (4 MAU, reserve)	1
Contingency	5
Total allowance	≈ 13

Assumptions: four users, 10,000 API requests per month at about 1 GB-second each, 1,000 of them needing model routing. The allowance is an engineering estimate, not an AWS Pricing Calculator quote. The account-wide budget is USD 50 per month; the 26 Sep snapshot showed USD 1.10 actual and USD 1.30 forecast. GitHub Actions minutes are billed separately by GitHub.

11 Testing and measured results

11.1 Automated tests

Suite	Scope	Result (27 Sep 2026)
Backend <code>pytest</code>	Sales, Inventory, Finance and Advisor logic; API contract and routing; Guard policy; audit ownership and approver permissions; DynamoDB compare-and-swap; Cognito token validation; Bedrock adapter failures; scenario expectations.	133 passed, 41 skipped (opt-in routing benchmark)
Frontend <code>test:dashboard</code>	Hive state rules and audit-event normalisation.	12 passed
Frontend <code>test:transport</code>	Concurrency, spacing and retry behaviour of the request scheduler.	12 passed
Frontend <code>test:finance</code>	Finance report view model.	4 passed
Frontend <code>test:contract</code>	Pinned backend OpenAPI routes, authentication and required fields.	Passed
Integration test	Vite proxy to local backend: Chinese evidence, Sales/Inventory routing, 528 overdue invoices, stored audit, server-controlled identity.	Passed

Suite	Scope	Result (27 Sep 2026)
BizzyData validator	21 CSVs, 498,535 records, scenario checks, byte-for-byte regeneration.	PASS
Container smoke + Trivy	Readiness and Sales/Inventory/Finance on packaged data, read-only root filesystem, 1 GB memory; HIGH/CRITICAL scan.	Passed; scan exit 0
Browser run (Playwright)	Flagship question in English and Chinese, approval of the pending action, all screenshots in this document.	0 page errors (with PR #9)

11.2 Success measures

Measure	Target	Result
Cross-functional review time	< 30 s for the demo scenario (baseline assumption ~10 min manual)	MET 0.29 s backend time for the flagship Sales + Inventory + Advisor question on the local build. AWS end-to-end latency including cold start is bounded by the 29 s Lambda timeout and was not benchmarked for this document.
Evidence coverage	100% of recommendations show source evidence	MET In all 9 measured questions, every agent that recommended an action also returned evidence; each item carries metric, value, period and source file (55 items for the flagship).
Routing efficiency	Only relevant agents invoked per intent	MET 7 of 9 questions routed exactly to the expected Bees and no irrelevant Bee was invoked. NOTE 2 phrasings (“complaints”, “stocking out”) missed the keyword rules and returned a clarification locally, where Bedrock is disabled.
Sensitive-action control	100% of tested AMBER/RED actions approved or blocked	MET 6/6 RED or unknown types blocked even when labelled GREEN; controlled types cannot be downgraded; invalid parameters are blocked; only approvers can decide.
Multilingual preservation	Critical values identical across languages	MET English and Chinese flagship questions returned the same routing, 55 evidence items and identical SGD values and IDs.

11.3 Routing and latency measurements

Question	Routed Bees	Guard decision	Evidence	Time
Why did sales fall this week?	Sales, Inventory	approval_required	55	290 ms
为什么这周销售下降了？	Sales, Inventory	approval_required	55	286 ms
How is revenue trending?	Sales	allowed	20	259 ms
Which invoices are overdue?	Finance	approval_required	12	205 ms
Which products are low on stock?	Inventory	approval_required	58	144 ms
客户投诉为什么增加？	Customer	approval_required	10	871 ms
Pay the supplier invoice now	Finance, Inventory	approval_required; no payment action exists	22	427 ms
Which products are at risk of stocking out?	none → clarification	allowed	0	3 ms
Why did customer complaints increase?	none → clarification	allowed	0	2 ms

Measured on 27 Sep 2026 against the local backend (backend main, packaged dataset, model provider disabled) on a developer laptop; times exclude network and Lambda cold start. Keyword rules match whole words, so plural or inflected forms can miss; in AWS those questions fall through to Bedrock.

12 Demo walkthrough: “Why did sales fall this week?”

Screenshots were taken on 27 Sep 2026 from the local build (backend `main` with the frontend fix in PR #9) using the synthetic dataset and a development identity. The deployed POC serves the same application behind Cognito sign-in.

Step 1 — Detect: the dashboard surfaces the problem

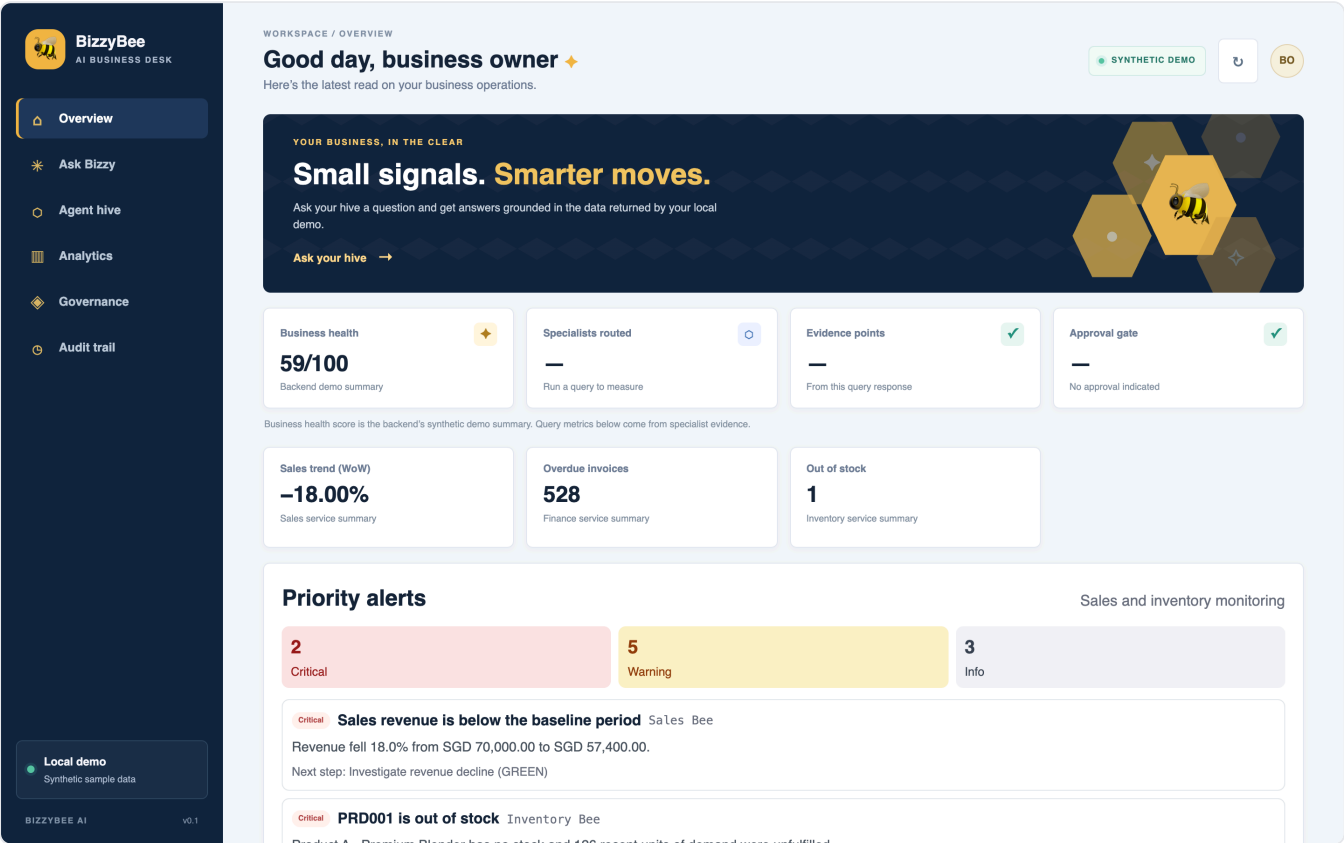


Figure 6. Overview: business health 59/100, sales -18.00% week-on-week, 528 overdue invoices, 1 product out of stock.

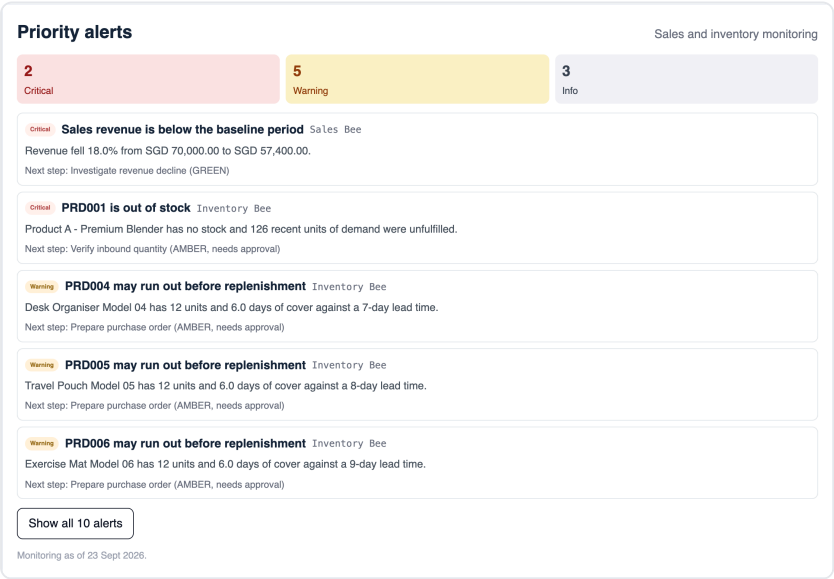


Figure 7. Priority alerts from Sales and Inventory Bees, each with the next step and its Guard level.

Step 2 — Specialist views are available without asking

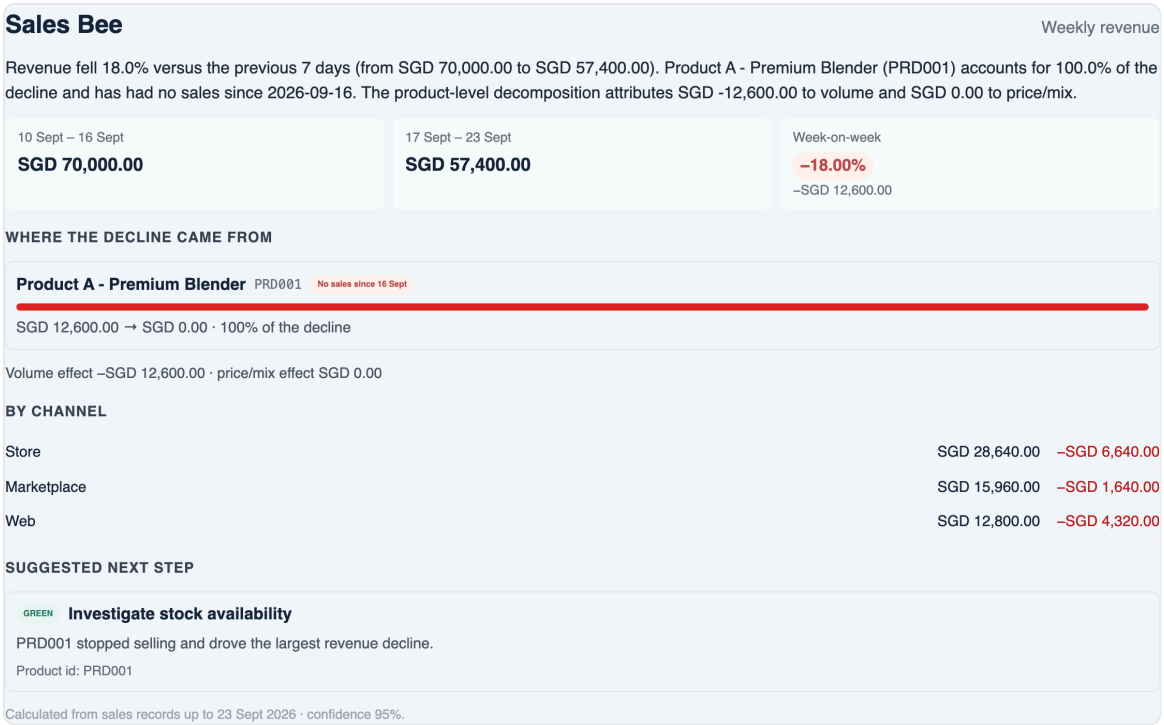


Figure 8. Sales Bee: PRD001 explains 100% of the decline, split by channel, with a GREEN next step.

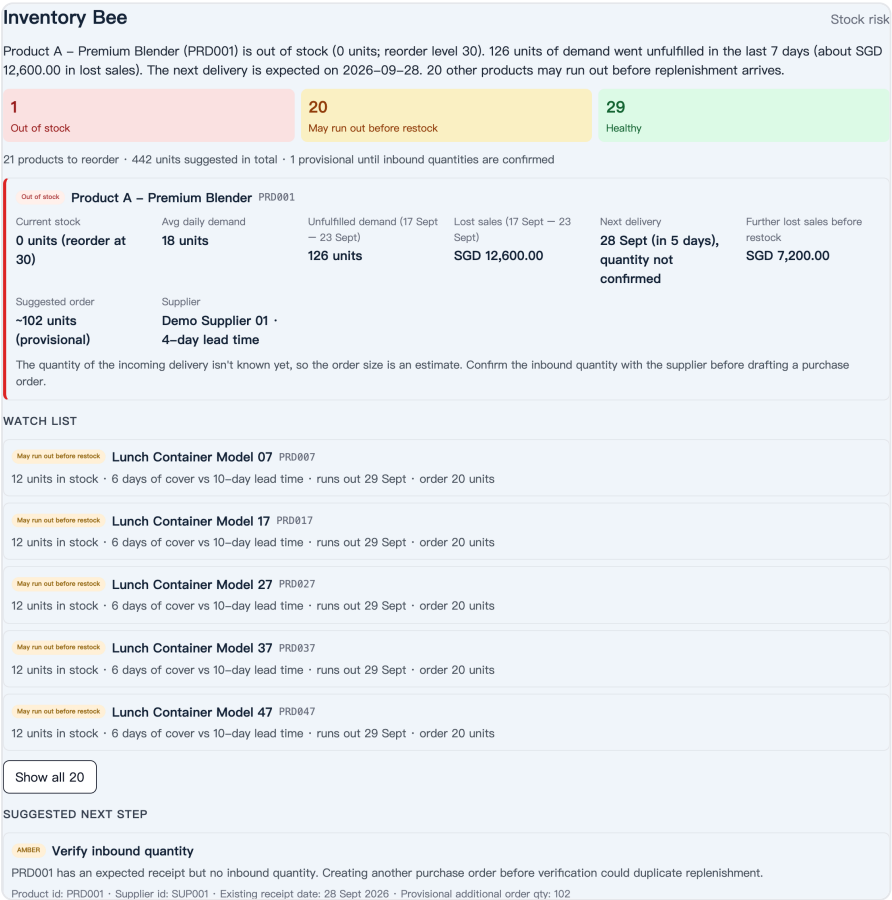


Figure 9. Inventory Bee: PRD001 out of stock, 126 units unfulfilled, delivery on 28 Sep with an unconfirmed quantity, so the suggested order is provisional and the next step is AMBER.

Finance Bee

Read-only synthetic SGD reports. Data ends 23 September 2026. Report summaries are currently in English.

Report

Profitability

Start date

2026/09/01

As of date

2026/09/23

Run Finance report

Profitability · 2026-09-01 to 2026-09-23 · SGD

From 2026-09-01 through 2026-09-23, net revenue is SGD 191,715.00, gross profit SGD 73,356.74, operating expenses SGD 20,417.86, and profit before tax SGD 52,938.88. Cash moved from SGD 3,255,606.66 to SGD 3,299,719.68; net cash flow is SGD 44,113.02. Figures are ledger-based, not model arithmetic.

Analysis intent
profitability

As of date
2026-09-23

Period start
2026-09-01

Period end
2026-09-23

Net revenue
\$191,715.00

Source: accounting_ledger.csv · Period: 2026-09-01/2026-09-23

Figure 10. Finance Bee profitability report (1-23 Sep): every figure is ledger-based with its source file and period.

Step 3 — Ask the hive

YOUR AI ASSISTANT

Ask BizzyBee

Local data · No external actions

Language

English

Your business question

Why did sales fall this week?

Ask the hive →

Critical values such as SGD, Invoice IDs and quantities stay structured regardless of language (English).

Last asked: Why did sales fall this week?

Figure 11. The owner asks one question; language is selectable and critical values stay structured.

SPECIALISTS AT WORK

Your agent hive

2 specialists routed

Queen Bee

Routed to sales, inventory

Success

Sales Bee

ROUTED

Revenue fell 18.0% versus the previous 7 days (from SGD 70,000.00 to SGD...

Success

Customer Bee

Not selected for this query

Idle

Finance Bee

Not selected for this query

Idle

Inventory Bee

ROUTED

Product A – Premium Blender (PRD001) is out of stock (0 units; reorder level 30), 126...

Success

Advisor Bee

The combined evidence strongly supports a stockout-related explanation for the sales...

Success

Guard Bee

approval_required

Success

Audit Bee

Trace 7bcef5f3-ebfa-4b31-ac84-0687a82358d9 · completed

Success

BEE WORKSPACE

Queen Bee

Routes each business question to relevant specialists.

Success

Routing decision

Why did sales fall this week?

2 selected

Sales Bee

Inventory Bee

Figure 12. Queen Bee routes to Sales and Inventory only; Customer and Finance stay idle; Guard and Audit report their state.

Step 4 — Evidence and correlation

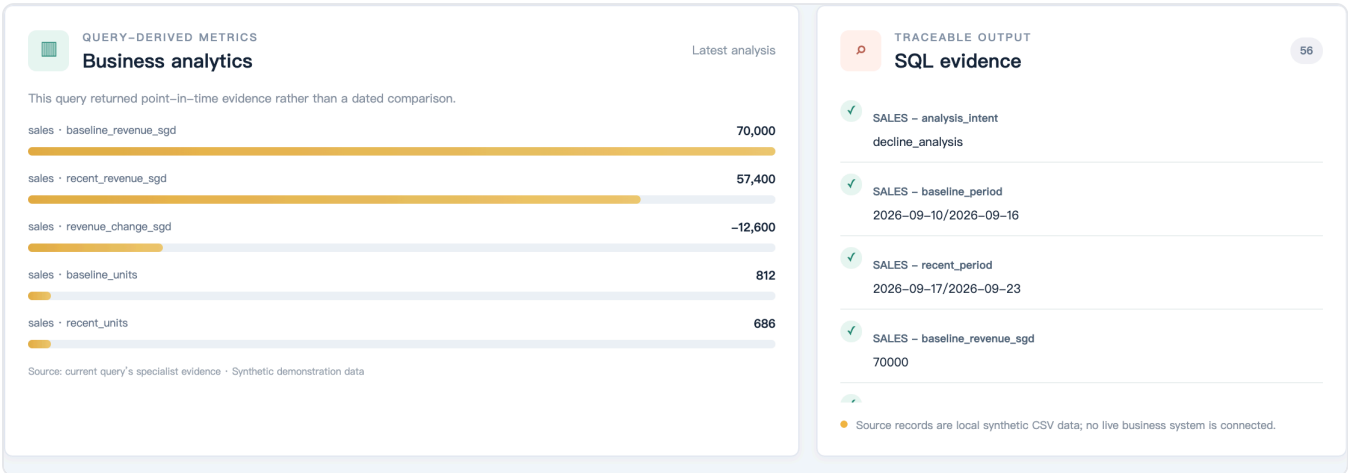


Figure 13. Query-derived metrics and the SQL evidence list (analysis intent, periods, baseline and recent revenue).

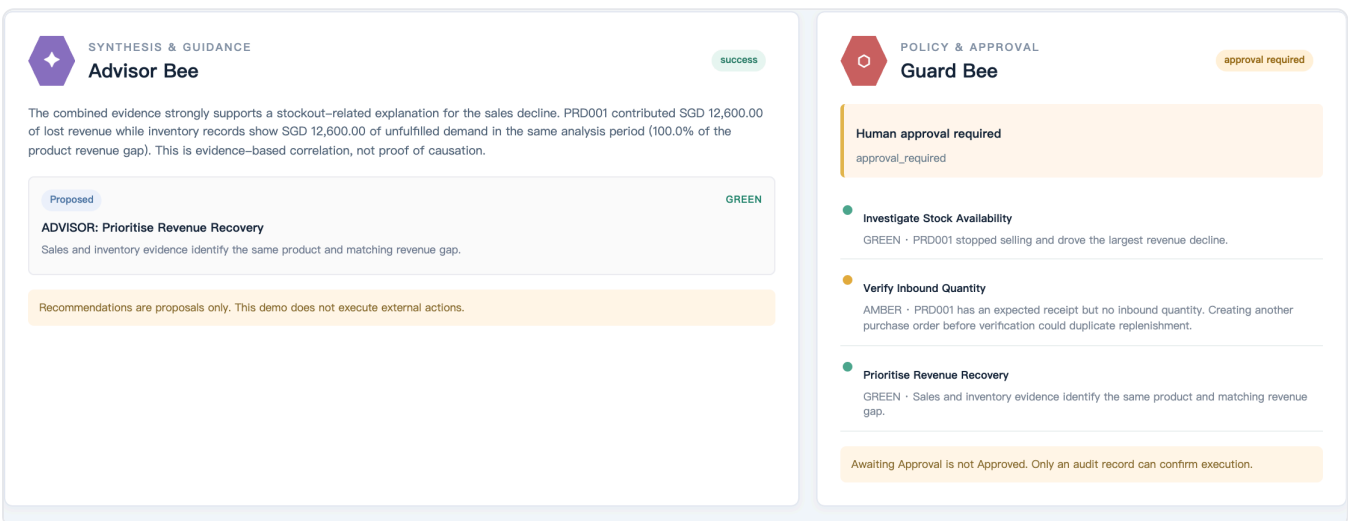


Figure 14. Advisor Bee links SGD 12,600 lost revenue with SGD 12,600 unfulfilled demand and says this is correlation, not proof of causation. Guard Bee lists each action: two GREEN, one AMBER.

Step 5 — Guard decision in the Bee workspace

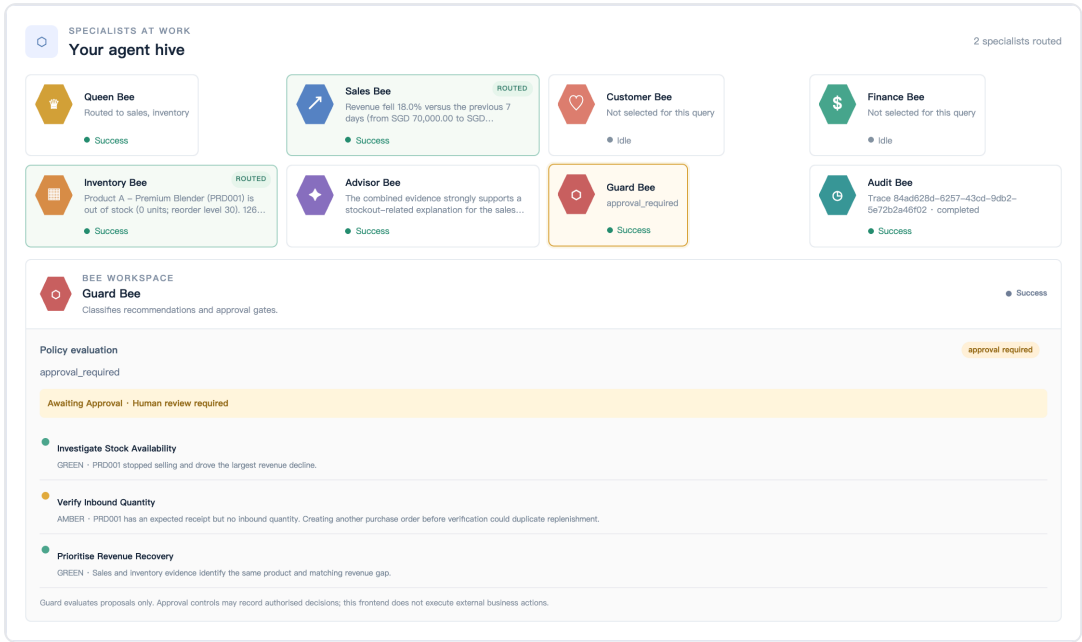


Figure 15. Guard Bee workspace: workflow requires approval; verifying the inbound quantity is AMBER because a duplicate purchase order could otherwise be created.

Step 6 — Approve and audit

The owner enters a reason and approves the AMBER draft (Figure 4 in section 6). The workflow event and its decision are stored in the audit table and can be inspected in the Audit Bee trace.

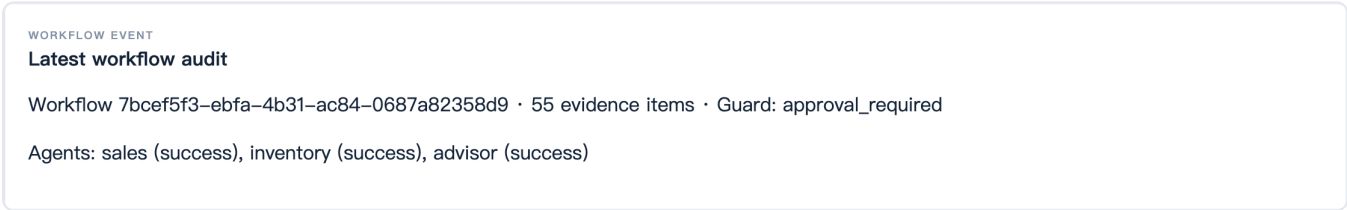


Figure 16. Latest workflow audit summary: 55 evidence items, Guard decision, agent statuses.

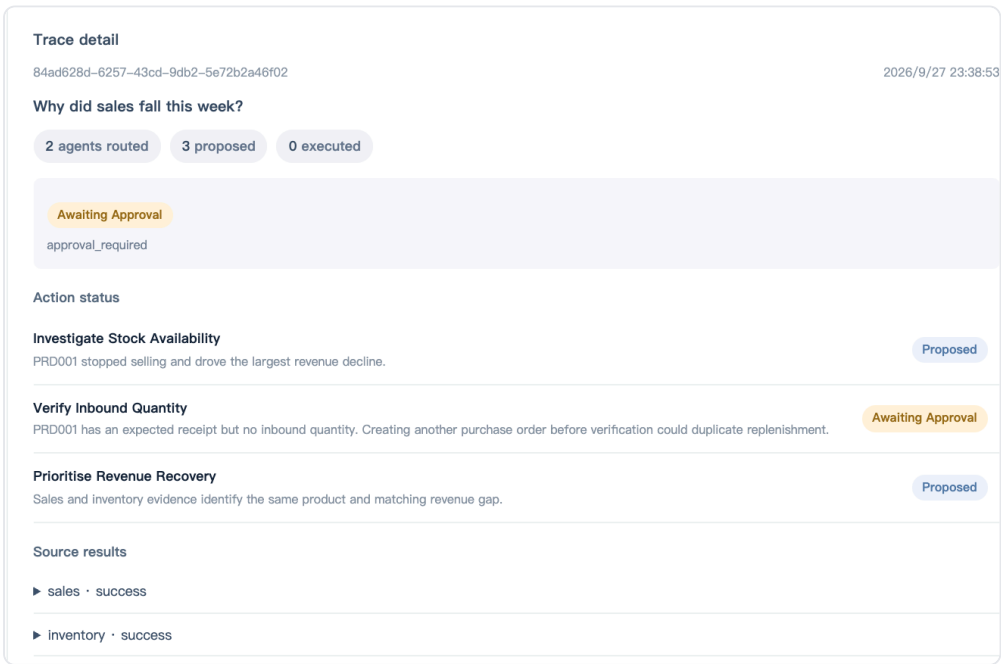


Figure 17. Audit Bee trace detail: 2 agents routed, 3 proposed actions, 0 executed, per-action status and source results.

Step 7 — Same answer in Chinese

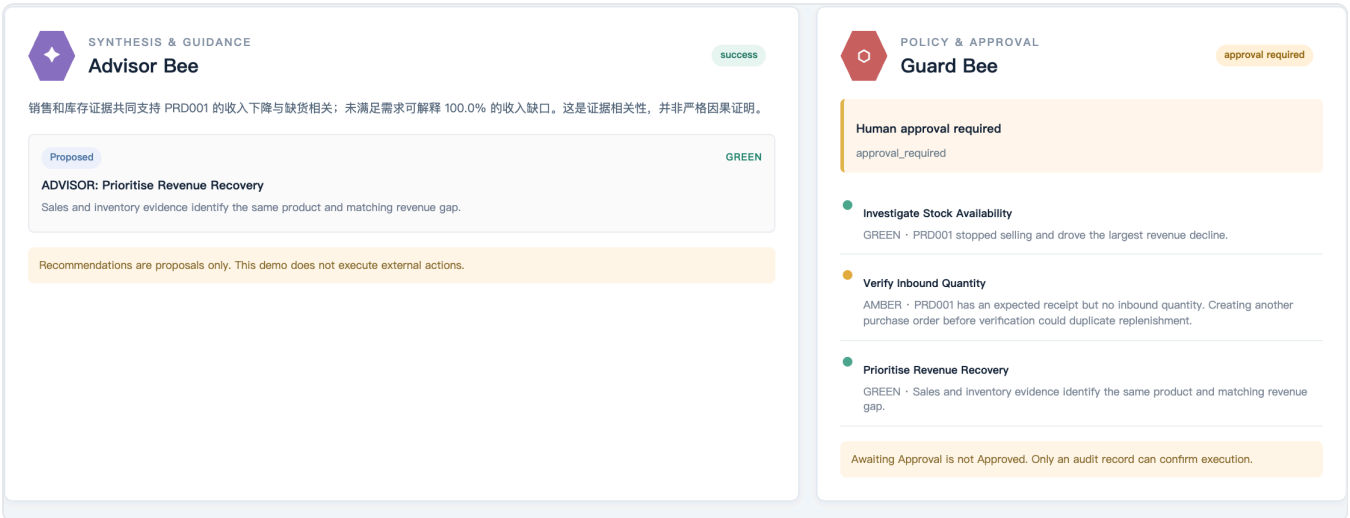


Figure 18. “为什么这周销售下降了？” returns a Chinese Advisor summary with the same SGD values, product ID and Guard classifications.

13 Known limitations and next steps

Limitation	Impact	Next step
Audit trace response shape differed from the frontend trace model	Before PR #9 the page went blank after each question.	Merge PR #9 (frontend-only mapping); agree one audit schema between backend and frontend.
GET /api/v1/audit list endpoint is not on backend main	“Recent traces” shows HTTP 404; the current trace still opens.	Add a paginated list by user and time (DynamoDB index) with the same read isolation.
Trace view does not refresh after a decision	An approved action still shows “Awaiting Approval” in the trace until reload.	Re-fetch the trace after approve/reject and show the <i>approved/rejected</i> state.
Keyword routing matches whole words only	Plural or inflected phrasing needs Bedrock or a clarification.	Add inflected forms and a routing regression set; keep Bedrock as fallback.
Localised output limited to English and Chinese	Malay, Tamil and Hindi are not yet claimed.	Add templates and tests per language before claiming support.
Continuous deployment disabled	Releases depend on the owner’s manual runbook.	Use a GitHub plan with environment reviewers, or an agreed alternative approval gate, then enable the OIDC deploy jobs.
Alarms have no notification channel	Incidents are only visible in the console.	Add an approved SNS or e-mail target.
Frontend placeholders without backend routes	/customers/opportunities and /finance/health are reserved in the client.	Implement or remove when the owners decide.
POC boundaries	Synthetic data, single tenant, no action executor, audit is not WORM.	Connectors, tenant-aware authorization, retention policy and an immutable archive for production.
Local tooling	Local Node 22 needs <code>--experimental-strip-types</code> for the TypeScript unit tests.	Use Node 24 locally, matching CI.

14 Team and ownership

Ownership as agreed in the Master Team Reference. All members integrated through the shared AgentResponse contract and pull requests into the BizzyBackend, BizzyFrontend and BizzyData repositories.

Member	Planned ownership	Planned deliverables
Yiting	Sales Bee + Customer Bee	Sales and customer tools, endpoints, evidence schema, unit tests, demo outputs for the sales decline and missed leads.
Sonne	Finance Bee + Inventory Bee	Invoice and finance calculations, stock-out and reorder logic, endpoints, unit tests, demo outputs for overdue invoices and stock risk.
Tiffany	Queen Bee + Advisor Bee	Intent routing, agent registry, orchestration, cross-agent synthesis, common query endpoint, end-to-end reasoning tests.
Malar	Guard Bee + Audit Bee + integration	GREEN/AMBER/RED policy engine, approval API, audit event model, trace view, security tests, integration checklist and demo coordination.

Implementation notes from repository history: the Inventory Bee analysis was built together with the Sales Bee (Yiting), including the sales-inventory health and alert APIs and their dashboard panels; the finance reports, AWS infrastructure, delivery controls, CI workflows and deployment were built by Sonne.

Key dates: final submission Monday 28 Sep 2026 · Finale Demo Day Saturday 10 Oct 2026 · Showcase and prize presentation Monday 19 Oct 2026.

A API reference

Method	Path	Purpose	Access
GET	/health, /ready	Liveness; readiness with dataset reporting date	Public
GET	/api/v1/me	Caller identity and approver flag	Signed-in user
GET	/api/v1/business-health	Health score and status	Signed-in user
GET	/api/v1/sales/summary	Sales Bee AgentResponse	Signed-in user
GET	/api/v1/inventory/status	Inventory Bee AgentResponse	Signed-in user
GET	/api/v1/finance/summary	Finance Bee AgentResponse	Signed-in user
GET	/api/v1/finance/report	Profitability and cash reports for a period	Signed-in user
GET	/api/v1/sales-inventory/health	Combined sales and inventory health	Signed-in user
GET	/api/v1/sales-inventory/alerts	Priority alert feed	Signed-in user
POST	/api/v1/query	Natural-language question → routed workflow result	Signed-in user
GET	/api/v1/audit/{workflow_id}	Stored workflow event	Creator or approver
POST	/api/v1/actions/{action_id}/approve	Approve a pending AMBER action with a reason	Approvers only
POST	/api/v1/actions/{action_id}/reject	Reject a pending action with a reason	Approvers only
Reserved in the frontend, not yet on backend main: GET /api/v1/audit?limit=, GET /api/v1/customers/opportunities, GET /api/v1/finance/health.			

B Sources and conventions

- Team documents: *Scope and Proposed Approach, Delivery, Measurement and Controls, Master Team Reference, Architecture & Delivery Plan* (23 Sep 2026) and the team's AWS deployment diagram.
- Repositories: BizzyBackend (`backend/` , `infra/` , `docs/deployment/` plan, completion and runtime status reports), BizzyFrontend (`src/` , `scripts/` , CI workflow) and BizzyData (generator and validator).
- Measurements in sections 11 and 12 were taken by the authors on 27 Sep 2026; AWS deployment facts are taken from the recorded Terraform applies and verification notes.
- Figures 2, 3 and 5 are redrawn for this document. AWS account, Cognito and API identifiers and user e-mail addresses are intentionally omitted.